

گزارش کار پروژه چهارم آزمایشگاه سیستم عامل

همگام سازی در xv6: شمارنده syscall، تولیدکننده - مصرف کننده، RW-lock و ticket lock

در این پروژه چهار خانواده از مکانیزم های همگام سازی در xv6 پیاده سازی و ارزیابی شد: شمارنده فراخوانی سیستمی در سه مدل بدون قفل، محافظت شده با `spinlock` و مختص هر پردازنده؛ مسئله تولیدکننده - مصرف کننده با بافر محدود داخل هسته و نسخه نهایی مبتنی بر `sleep`/`wakeup`؛ قفل خواننده - نویسنده با سیاست حق تقدم نویسنده؛ و قفل بلیطی با تضمین ترتیب FIFO برای ورود به ناحیه بحرانی.

نویسندگان

معراج رستگار - 810102576

علی صادقی - 810102471

معراج پورحسینی - 810102420

۱ خلاصه اجرایی

۱-۱ فهرست تغییرات کد

۲ بخش اول: مشاهده اثر تک‌هسته‌ای و چندهسته‌ای در همگام‌سازی

۲-۱ پاسخ سوالات تئوری بخش اول

- ۲-۱-۱ سوال ۱ – مسیر کنترلی در xv6 چیست؟
- ۲-۱-۲ سوال ۲ – هسته با ورود مجدد چیست؟
- ۲-۱-۳ سوال ۳ – تفاوت متن پردازش و متن وقفه
- ۲-۱-۴ سوال ۴ – چرا `syscall_count[num]++` اتمیک نیست؟
- ۲-۱-۵ سوال ۵ – چرا نسخه بدون قفل در تک‌هسته‌ای معمولاً درست دیده می‌شود؟
- ۲-۱-۶ سوال ۶ – چگونه `spinlock` محافظت می‌کند؟
- ۲-۱-۷ سوال ۷ – چرا `xv6` هنگام گرفتن `spinlock` وقفه را غیرفعال می‌کند؟
- ۲-۱-۸ سوال ۸ – وقفه قابل ماسک و غیرقابل ماسک
- ۲-۱-۹ سوال ۹ – تفاوت `cli/sti` مستقیم با `pushcli/popcli`
- ۲-۱-۱۰ سوال ۱۰ – اثر شمارنده سراسری روی `cache` در چندهسته‌ای

۲-۲ پیاده‌سازی عملی بخش اول

- ۲-۲-۱ رابط `syscall`ها
- ۲-۲-۲ سه مدل شمارنده
- ۲-۲-۳ نقطه ثبت شمارنده
- ۲-۲-۴ دلیل استفاده از `pushcli/popcli` در نسخه `per-CPU`
- ۲-۲-۵ برنامه تست

۳ بخش دوم: پیاده‌سازی تولیدکننده - مصرف‌کننده در هسته xv6

۳-۱ پاسخ سوالات تئوری بخش دوم

- ۳-۱-۱ سوال ۱ – مسئله تولیدکننده - مصرف‌کننده
- ۳-۱-۲ سوال ۲ – تفاوت `sleep` و `spinlock`
- ۳-۱-۳ سوال ۳ – انتظار فعال چیست؟
- ۳-۱-۴ سوال ۴ – چرا `producer/consumer` نباید فعالانه منتظر بمانند؟

۳-۲ پیاده‌سازی عملی بخش دوم

- ۳-۲-۱ ساختار بافر
- ۳-۲-۲ نسخه نهایی: `blocking` با `sleep/wakeup`
- ۳-۲-۳ رابط `syscall`ها
- ۳-۲-۴ برنامه تست

۴ بخش سوم: قفل خواننده - نویسنده و جلوگیری از گرسنگی نویسنده

۴-۱ پاسخ سوالات تئوری بخش سوم

۴-۱-۱ سوال ۱ - مسئله خوانندگان و نویسندگان

۴-۱-۲ سوال ۲ - گرسنگی نویسنده در قفل ساده

۴-۱-۳ سوال ۳ - حق تقدم نویسنده

۴-۱-۴ سوال ۴ - نیاز به spinlock داخلی

۴-۲ پیاده‌سازی عملی بخش سوم

۴-۲-۱ ساختار قفل

۴-۲-۲ سیاست ورود reader

۴-۲-۳ سیاست ورود writer

۴-۲-۴ رابط syscalls

۴-۲-۵ برنامه تست

۵ بخش چهارم: قفل بلیطی و عدالت

۵-۱ پاسخ سوالات تئوری بخش چهارم

۵-۱-۱ سوال ۱ - نحوه کار قفل بلیطی

۵-۱-۲ سوال ۲ - تفاوت عدالت spinlock استاندارد و ticket lock

۵-۱-۳ سوال ۳ - مقایسه priority lock و ticket lock

۵-۱-۴ سوال ۴ - مشکل عملکردی ticket lock در چندهسته‌ای

۵-۱-۵ سوال ۵ - هدف spinlock داخلی در ticket lock

۵-۲ پیاده‌سازی عملی بخش چهارم

۵-۲-۱ ساختار قفل

۵-۲-۲ گرفتن قفل

۵-۲-۳ آزادسازی قفل

۵-۲-۴ رابط syscalls

۵-۲-۵ برنامه تست

۵-۳ راهنمای اجرا

۵-۳-۱ build

۵-۳-۲ اجرای xv6 با تعداد CPU متفاوت

۵-۳-۳ اجرای تست‌ها داخل xv6

۶ نکات طراحی و فرض‌های پیاده‌سازی

۷ نتیجه‌گیری

خلاصه اجرایی

در این پروژه چهار خانواده مکانیزم همگام‌سازی در xv6 پیاده‌سازی شد:

1. شمارنده فراخوانی سیستمی با سه مدل طراحی:

- شمارنده سراسری بدون قفل؛
- شمارنده سراسری محافظت‌شده با spinlock؛
- شمارنده‌های مختص هر پردازنده، به‌عنوان نسخه نهایی و پیش‌فرض.

2. مسئله تولیدکننده - مصرف‌کننده با بافر محدود داخل هسته و نسخه نهایی مبتنی بر wakeup / sleep برای حذف انتظار فعال.

3. قفل خواننده - نویسنده با سیاست حق تقدم نویسنده برای جلوگیری از گرسنگی نویسنده.

4. قفل بلیطی با تضمین ترتیب FIFO برای ورود به ناحیه بحرانی.

پیاده‌سازی نهایی از نظر build و بسته‌بندی تحویل با دستورهای زیر اعتبارسنجی شد:

BASH

```
make clean
make
make fs.img
make dist
cd dist && make fs.img
```

نکته بازبینی نهایی: build کامل kernel، برنامه‌های سطح کاربر، fs.img و build مجدد از dist موفق بوده است. اجرای واقعی CPU=4 نیز پس از اصلاح topology QEMU انجام شد و خروجی آن در ضمیمه نهایی همین گزارش آمده است.

فهرست تغییرات کد

فایل	تغییرات اصلی
param.h	تعریف NSYSCALL و حالت‌های شمارنده syscall بدون قفل، با قفل، و per-CPU.
syscall.h	افزودن شماره syscall‌های جدید Lab4.
syscall.c	افزودن زیرساخت شمارش syscall‌ها و ثبت شمارنده در مسیر مرکزی syscall().
sysproc.c	پیاده‌سازی producer-consumer، reader-writer lock، ticket lock و wrapperهای syscall.
defs.h	افزودن prototype‌های مقداردهی اولیه و شمارنده syscall.

فایل	تغییرات اصلی
main.c	فراخوانی syscallscountinit() و lab4syncinit() هنگام boot.
user.h	افزودن prototype های فضای کاربر.
usys.S	افزودن stub های assembly برای syscalls های جدید.
Makefile	افزودن برنامه های تست به UPROGS و EXTRA برای حفظ صحت make dist.
scounttest.c	تست شمارنده syscalls و شمارنده های per-CPU.
pctest.c	تست تولیدکننده - مصرف کننده.
rwtest.c	تست قفل خواننده - نویسنده با حق تقدم نویسنده.
tickettest.c	تست قفل بلیطی و ترتیب FIFO.

بخش اول: مشاهده اثر تک هسته‌ای و چند هسته‌ای در همگام‌سازی

پاسخ سوالات تئوری بخش اول

سوال 1 – مسیر کنترلی در xv6 چیست؟

مسیر کنترلی یا **control path** دنباله‌ای از اجرای کد داخل هسته است که در پاسخ به یک رویداد وارد **kernel mode** می‌شود. در xv6 سه مسیر اصلی داریم:

- **فراخوانی سیستمی:** پردازش کاربر با دستور **trap** وارد هسته می‌شود و سرویس مشخصی مانند **read**، **write** یا **getpid** را درخواست می‌کند.
 - **وقفه:** رویدادی ناهمگام از سخت‌افزار است؛ مثل وقفه **timer**، دیسک یا صفحه‌کلید. وقفه الزاماً به دستور جاری پردازش وابسته نیست.
 - **استثنا:** رخدادی همگام با اجرای دستور جاری است؛ مثل **page fault**، تقسیم بر صفر یا دستور نامعتبر.
- تفاوت اصلی این سه در منشأ رخداد، زمان‌بندی رخداد، و انتظارات هسته از ادامه اجرای پردازش است.

سوال 2 – هسته با ورود مجدد چیست؟

هسته **reentrant** هسته‌ای است که چند مسیر کنترلی می‌توانند هم‌زمان یا تو در تو وارد آن شوند. برای مثال، یک پردازش در حال اجرای **syscall** است و هم‌زمان **timer interrupt** رخ می‌دهد. **handler** وقفه نیز وارد کد هسته می‌شود. اگر هر دو مسیر به داده مشترکی مانند جدول پردازش‌ها، شمارنده‌ها یا بافرها دسترسی داشته باشند، بدون قفل ممکن است **race condition** رخ دهد.

سوال 3 – تفاوت متن پردازش و متن وقفه

متن پردازش یعنی کدی که از طرف یک پردازش مشخص اجرا می‌شود و می‌تواند در صورت نیاز **block** یا **sleep** شود. **متن وقفه** مستقل از منطق عادی پردازش و در پاسخ به سخت‌افزار اجرا می‌شود. کد متن وقفه نباید **sleep** شود، چون معمولاً پردازش مشخصی برای خواباندن وجود ندارد و نگه‌داشتن مسیر وقفه می‌تواند باعث **deadlock**، افزایش **latency** وقفه‌ها، و توقف پیشرفت کل سیستم شود.

سوال 4 – چرا `syscall_count[num]++` اتمیک نیست؟

عبارت `++` در سطح ماشین معمولاً به چند گام شکسته می‌شود:

1. خواندن مقدار از حافظه؛
2. افزایش مقدار در ثبات؛
3. نوشتن مقدار جدید در حافظه.

زمان	CPU0	CPU1
t1	مقدار 10 را می‌خواند	
t2		مقدار 10 را می‌خواند
t3	مقدار را به 11 تبدیل می‌کند	مقدار را به 11 تبدیل می‌کند
t4	11 را می‌نویسد	11 را می‌نویسد

در حالی که دو increment انجام شده، مقدار نهایی به جای 12 برابر 11 می‌شود. این همان **lost update** است.

سوال 5 – چرا نسخه بدون قفل در تک‌هسته‌ای معمولاً درست دیده می‌شود؟

در تک‌هسته‌ای، اجرای واقعی هم‌زمان روی دو CPU وجود ندارد. چند پردازش می‌توانند هم‌روند باشند، اما در هر لحظه فقط یک CPU کد هسته را اجرا می‌کند. اگر context switch دقیقاً وسط عملیات بحرانی رخ ندهد یا interrupt مسیر مشابهی را اجرا نکند، خطا مشاهده نمی‌شود. اما در چندهسته‌ای، دو CPU می‌توانند واقعاً هم‌زمان همان شمارنده را بخوانند و بنویسند؛ بنابراین احتمال **lost update** واقعی ایجاد می‌شود.

سوال 6 – spinlock چگونه محافظت می‌کند؟

spinlock با استفاده از عملیات اتمیک سخت‌افزاری مالکیت قفل را به یک CPU محدود می‌کند. هر CPU پیش از ورود به critical section قفل را می‌گیرد و پس از پایان آن را آزاد می‌کند. بنابراین در نسخه سراسری قفل‌دار، تنها یک CPU در هر لحظه `syscall_count[num]++` را انجام می‌دهد و **lost update** حذف می‌شود. نقطه ضعف این روش، رقابت همه CPUها روی یک cache line و یک قفل مشترک است.

سوال 7 – چرا هنگام گرفتن spinlock وقفه را غیرفعال می‌کند؟

در xv6، `acquire()` با `pushcli()` وقفه‌های CPU جاری را غیرفعال می‌کند. دلیل اصلی جلوگیری از deadlock بازگشتی است. سناریو:

1. پردازش روی CPU0 قفل L را می‌گیرد.
 2. قبل از آزاد کردن L، وقفه timer یا device روی همان CPU رخ می‌دهد.
 3. handler وقفه نیز تلاش می‌کند L را بگیرد.
 4. چون L در اختیار همان CPU است و handler منتظر آزاد شدن آن می‌چرخد، CPU خودش را deadlock می‌کند.
- پس خاموش کردن وقفه‌ها در بازه نگه‌داشتن spinlock ضروری است.

سوال 8 – وقفه قابل ماسک و غیرقابل ماسک

وقفه قابل ماسک وقفه‌ای است که CPU/OS می‌تواند موقتاً آن را غیرفعال کند. وقفه غیرقابل ماسک یا NMI برای رخدادهای بحرانی مثل خطای سخت‌افزاری جدی استفاده می‌شود و سیستم‌عامل نمی‌تواند آن را دلخواه خاموش کند. چون برخی رخدادهای حیاتی‌اند، کد مربوط به آن‌ها باید کوتاه، سریع و بدون عملیات blocking باشد.

سوال 9 – تفاوت cli/sti مستقیم با pushcli/popcli

cli و sti مستقیماً وقفه را خاموش/روشن می‌کنند. اما pushcli/popcli در xv6 تودرتویی را مدیریت می‌کند. اگر دو critical section تو در تو داشته باشیم، یک sti مستقیم در خروج از بخش داخلی می‌تواند وقفه را زودتر از موعد فعال کند؛ در حالی‌که هنوز بخش خارجی critical section است. pushcli/popcli با شمارنده ncli فقط وقتی وقفه را فعال می‌کند که تمام لایه‌های critical section خارج شده باشند.

سوال 10 – اثر شمارنده سراسری روی cache در چند هسته‌ای

شمارنده سراسری مشترک باعث می‌شود همه CPUها یک cache line مشترک را مکرراً invalidate و acquire کنند. در پروتکل‌های cache coherency، نوشتن یک CPU روی cache line مشترک باعث نامعتبر شدن نسخه سایر CPUها می‌شود. شمارنده‌های per-CPU این مشکل را کم می‌کنند، چون هر CPU عمدتاً فقط خانه خودش را می‌نویسد و رقابت نوشتاری روی یک داده مشترک حذف می‌شود. جمع کل فقط هنگام گزارش‌گیری انجام می‌شود.

پیاده‌سازی عملی بخش اول

رابط syscallها

دو syscall اضافه شد:

C

```
int getcount(int syscall_number);
int getcpucount(int cpu_id, int syscall_number);
```

سه مدل شمارنده

در param.h سه حالت تعریف شده است:

C

```
#define SYSCALL_COUNT_GLOBAL_UNLOCKED 0
#define SYSCALL_COUNT_GLOBAL_LOCKED 1
#define SYSCALL_COUNT_PERCPU 2
#define SYSCALL_COUNT_MODE SYSCALL_COUNT_PERCPU
#define NSYSCALL 64
```


نسخه نهایی روی `SYSCALL_COUNT_PERCPU` تنظیم شده است.

نقطه ثبت شمارنده

در `syscall.c` ، داخل مسیر مرکزی `syscall()` ، قبل از اجرای handler مربوطه شمارنده ثبت می‌شود:

C

```
if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
    syscall_count_record(num);
    curproc->tf->eax = syscalls[num]();
}
```

دلیل استفاده از `pushcli/popcli` در نسخه per-CPU

برای استفاده از `cpuid()` در `xv6` باید وقفه‌ها روی CPU جاری خاموش باشند؛ زیرا در غیر این صورت پردازش ممکن است بعد از خواندن CPU و قبل از `increment` جابه‌جا شود. بنابراین در نسخه per-CPU:

C

```
pushcli();
id = cpuid();
syscall_count_cpu[id][num]++;
popcli();
```

برنامه تست

برنامه `scounttest.c` چند فرزند ایجاد می‌کند و هر فرزند تعداد زیادی `getpid()` اجرا می‌کند. سپس مقدار کل و مقدار هر CPU چاپ می‌شود.

اجرای پیشنهادی داخل `xv6`:

BASH

```
scounttest 8 10000
```

نمونه خروجی مورد انتظار:

TEXT

```
syscall counter test
children: 8 iterations: 10000
CPU 0 getpid count: 20130
CPU 1 getpid count: 19870
CPU 2 getpid count: 20010
CPU 3 getpid count: 19990
CPU 4 getpid count: 0
CPU 5 getpid count: 0
CPU 6 getpid count: 0
CPU 7 getpid count: 0
Total getpid count: 80000
Expected count: 80000
```

در محیطی با CPUS=1 انتظار می‌رود تقریباً تمام شمارنده روی CPU0 باشد. در محیطی با CPUS=4 توزیع بین CPUهای فعال دیده می‌شود.

بخش دوم: پیاده‌سازی تولیدکننده - مصرف‌کننده در هسته xv6

پاسخ سوالات تئوری بخش دوم

سوال 1 – مسئله تولیدکننده - مصرف‌کننده

در این مسئله یک یا چند تولیدکننده داده را در یک بافر محدود قرار می‌دهند و یک یا چند مصرف‌کننده داده را از بافر خارج می‌کنند. چون بافر ظرفیت محدود دارد، تولیدکننده هنگام پر بودن بافر و مصرف‌کننده هنگام خالی بودن بافر باید منتظر بمانند. علاوه بر آن، متغیرهایی مانند `head`، `tail`، `count` و آرایه بافر مشترک‌اند و باید محافظت شوند.

سوال 2 – تفاوت `sleep` و `spinlock`

`spinlock` برای `critical section` کوتاه مناسب است؛ چون پردازنده/CPU هنگام انتظار فعالانه می‌چرخد. اما برای انتظار طولانی، مثل انتظار برای خالی‌شدن یا پرشدن بافر، `spinlock` مناسب نیست؛ چون CPU را بدون انجام کار مفید مصرف می‌کند. `sleep` پردازنده را از صف اجرا خارج می‌کند تا با `wakeup` در زمان مناسب بیدار شود.

سوال 3 – انتظار فعال چیست؟

انتظار فعال یعنی پردازنده در حلقه‌ای مداوم شرطی را چک کند، بدون این‌که کار مفید انجام دهد. مثلاً مصرف‌کننده دائماً `consume()` را صدا بزند تا شاید مقدار غیر 1- بگیرد. این روش CPU را مصرف می‌کند و باعث کاهش `throughput` سیستم می‌شود.

سوال 4 – چرا `producer/consumer` نباید فعالانه منتظر بمانند؟

اگر بافر پر است، تولیدکننده تا آزاد شدن خانه جدید کاری نمی‌تواند انجام دهد. اگر بافر خالی است، مصرف‌کننده تا تولید داده جدید کاری نمی‌تواند انجام دهد. در هر دو حالت، اجرای حلقه `retry` فقط CPU را تلف می‌کند. راه درست خوابیدن روی کانال مناسب و بیدارشدن پس از تغییر وضعیت بافر است.

پیاده‌سازی عملی بخش دوم

ساختار بافر

در `sysproc.c` ساختار زیر پیاده‌سازی شد:

C

```
#define PC_BUFFER_SIZE 16

struct pc_buffer {
    int buffer[PC_BUFFER_SIZE];
    int head;
    int tail;
    int count;
    int not_full;
    int not_empty;
    struct spinlock lock;
};
```

نسخه نهایی: blocking با sleep/wakeup

در نسخه نهایی، produce هنگام پر بودن بافر روی کانال not_full می‌خوابد و consume هنگام خالی بودن بافر روی کانال not_empty می‌خوابد.

الگوی کلی:

C

```
while(pcbuf.count == PC_BUFFER_SIZE)
    sleep(&pcbuf.not_full, &pcbuf.lock);

while(pcbuf.count == 0)
    sleep(&pcbuf.not_empty, &pcbuf.lock);
```

برای جلوگیری از lost wakeup، بررسی شرط و خوابیدن با همان قفل بافر انجام می‌شود.

رابط syscalls

C

```
int produce(int value);
int consume(void);
```

برنامه تست

برنامه pctest.c سه producer و سه consumer می‌سازد. consumerها زودتر شروع می‌شوند تا ابتدا روی بافر خالی بخوابند؛ سپس producerها داده تولید می‌کنند و consumerها بیدار می‌شوند.

اجرای پیشنهادی:

BASH

pctest

TEXT

```
producer-consumer test: blocking sleep/wakeup version
producer 1 produced 100
consumer 1 consumed 100
producer 2 produced 200
consumer 2 consumed 200
producer 3 produced 300
consumer 3 consumed 300
...
producer-consumer test done
```

تحلیل: `buffer` ، `head` ، `tail` و `count` داده مشترک هستند؛ بنابراین همه دسترسی‌ها با `pcbuf.lock` محافظت شده‌اند. اما انتظار برای تغییر وضعیت بافر با `spin` انجام نشده و پردازش‌ها واقعاً `sleep` می‌شوند.

بخش سوم: قفل خواننده - نویسنده و جلوگیری از گرسنگی نویسنده

پاسخ سوالات تئوری بخش سوم

سوال 1 – مسئله خوانندگان و نویسندگان

در منابعی که read-heavy هستند، چند خواننده می‌توانند هم‌زمان وارد critical section شوند، چون خواندن داده را تغییر نمی‌دهد. اما نویسنده باید دسترسی انحصاری داشته باشد. اگر برای همه عملیات از قفل انحصاری معمولی استفاده شود، خواننده‌ها نیز بی‌دلیل serialize می‌شوند و کارایی کاهش می‌یابد.

سوال 2 – گرسنگی نویسنده در قفل ساده

در قفل خواننده - نویسنده ساده، اگر خواننده‌های جدید همیشه اجازه ورود داشته باشند، ممکن است هیچ‌وقت تعداد خوانندگان به صفر نرسد. در این حالت نویسنده‌ای که منتظر است دائماً عقب می‌افتد و دچار starvation می‌شود.

سوال 3 – حق تقدم نویسنده

در سیاست writer priority، وقتی حداقل یک نویسنده منتظر است، خواننده‌های جدید اجازه ورود نمی‌گیرند. خواننده‌های فعلی کارشان را تمام می‌کنند؛ سپس نویسنده وارد می‌شود. این سیاست گرسنگی نویسنده را حذف می‌کند، اما اگر جریان نویسندگان پیوسته باشد، امکان انتظار طولانی یا حتی گرسنگی خوانندگان وجود دارد.

سوال 4 – نیاز به spinlock داخلی

متغیرهایی مانند `readers`، `active_writer` و `waiting_writers` خودشان داده مشترک‌اند. اگر چند CPU هم‌زمان این وضعیت را بخوانند/بنویسند، خود قفل خواننده - نویسنده خراب می‌شود. بنابراین یک `spinlock` داخلی برای محافظت از state قفل لازم است.

پیاده‌سازی عملی بخش سوم

ساختار قفل

در `sysproc.c`:

C

```
struct rwlock {
    int readers;
    int active_writer;
    int waiting_writers;
    int writer_pid;
    int reader_chan;
    int writer_chan;
    struct spinlock lock;
};
```

سیاست ورود reader

reader فقط وقتی وارد می‌شود که:

C

```
active_writer == 0 && waiting_writers == 0
```

این شرط باعث می‌شود خواننده‌ای که بعد از درخواست نویسنده آمده، حتی اگر خواننده‌های قدیمی هنوز فعال باشند، وارد نشود.

سیاست ورود writer

writer ابتدا `waiting_writers` را زیاد می‌کند و سپس تا وقتی writer فعال یا reader فعال وجود دارد، می‌خوابد:

C

```
rw->waiting_writers++;
while(rw->active_writer || rw->readers > 0)
    sleep(&rw->writer_chan, &rw->lock);
```

رابط syscallsها

C

```
int rw_acquire_read(void);
int rw_release_read(void);
int rw_acquire_write(void);
int rw_release_write(void);
```

برنامه تست

سناریوی `rwtest.c`:

1. چند reader اولیه وارد می‌شوند و مدتی در critical section می‌مانند.
2. یک writer درخواست ورود می‌دهد.

3. چند reader جدید بعد از writer درخواست می‌دهند.

4. readerهای جدید نباید قبل از writer وارد شوند.

اجرای پیشنهادی:

BASH

rwtest

نمونه خروجی مورد انتظار:

TEXT

```
reader-writer lock test: writer priority
early reader 1 ENTERS
early reader 2 ENTERS
early reader 3 ENTERS
writer requests write lock
late reader 1 requests read lock
late reader 2 requests read lock
late reader 3 requests read lock
early reader 1 leaves
early reader 2 leaves
early reader 3 leaves
writer ENTERS
writer leaves
late reader 1 ENTERS after writer priority check
late reader 2 ENTERS after writer priority check
late reader 3 ENTERS after writer priority check
reader-writer test done
```

تحلیل: late reader پس از writer درخواست داده‌اند، اما تا پایان writer وارد نشده‌اند؛ بنابراین writer starvation رفع شده است.

بخش چهارم: قفل بلیطی و عدالت

پاسخ سوالات تئوری بخش چهارم

سوال 1 – نحوه کار قفل بلیطی

قفل بلیطی دو شمارنده دارد: `ticket` برای توزیع شماره جدید و `turn` برای شمارهای که اکنون اجازه ورود دارد. هر پدازه هنگام درخواست قفل یک بلیط منحصر به فرد می‌گیرد و تا وقتی `turn == myticket` نشود منتظر می‌ماند. هنگام آزادسازی، مالک قفل `turn` را یک واحد زیاد می‌کند.

سوال 2 – تفاوت عدالت `spinlock` استاندارد و `ticket lock`

در `spinlock` استاندارد `xv6`، هر CPU که زودتر در رقابت سخت‌افزاری موفق شود قفل را می‌گیرد؛ ترتیب FIFO تضمین نمی‌شود. در `ticket lock`، ترتیب ورود بر اساس شماره بلیط است و FIFO تضمین می‌شود.

سوال 3 – مقایسه `ticket lock` و `priority lock`

معیار	Priority Lock	Ticket Lock
عدالت	وابسته به سیاست <code>priority</code> ؛ ممکن است برای گروه کم‌اولویت ناعادلانه باشد.	FIFO و بسیار عادلانه.
پیچیدگی	به مدیریت <code>priority</code> و صف/شرط‌های بیشتر نیاز دارد.	ساده‌تر: دو شمارنده و یک قفل داخلی برای گرفتن بلیط.
گرسنگی	اگر <code>priority</code> سخت اعمال شود، <code>starvation</code> ممکن است.	در حالت عادی <code>starvation</code> ندارد، چون نوبت‌ها <code>monotonically</code> جلو می‌روند.

سوال 4 – مشکل عملکردی `ticket lock` در چند هسته‌ای

اگرچه `ticket lock` عادلانه است، همه منتظرها روی متغیر مشترک `spin turn` می‌کنند. در معماری‌های چند هسته‌ای و پروتکل‌هایی مانند MESI، تغییر `turn` باعث `invalidate/read traffic` روی `cache line` مشترک می‌شود. با افزایش تعداد هسته‌ها، همه CPUهای منتظر مرتباً همان `cache line` را می‌خوانند و آزادسازی قفل باعث موجی از `coherence traffic` می‌شود.

سوال 5 – هدف `spinlock` داخلی در `ticket lock`

عملیات گرفتن بلیط باید اتمیک باشد. اگر دو CPU هم‌زمان `ticket` را بخوانند و بدون قفل زیاد کنند، ممکن است هر دو یک شماره بگیرند. این شرط `race` و FIFO و انحصار را می‌شکند. بنابراین در این پیاده‌سازی یک `spinlock` داخلی فقط برای توزیع امن بلیط استفاده شده است.

پیاده‌سازی عملی بخش چهارم

ساختار قفل

در sysproc.c :

C

```
struct ticketLock {
    volatile uint ticket;
    volatile uint turn;
    int owner_pid;
    uint owner_ticket;
    struct spinlock lock;
};
```

volatile برای ticket و turn استفاده شده تا loop انتظار مقدار turn را دوباره از حافظه بخواند.

گرفتن قفل

1. با قفل داخلی، بلیط اختصاص داده می‌شود.
2. پیام kernel-level با cprintf چاپ می‌شود.
3. پردازش تا رسیدن نوبتش منتظر می‌ماند.
4. پس از ورود، مالک ثبت می‌شود.

C

```
acquire(&tl->lock);
myticket = tl->ticket;
tl->ticket++;
release(&tl->lock);

while(tl->turn != myticket)
    ;
```

آزادسازی قفل

در release، مالکیت بررسی می‌شود و سپس turn یک واحد افزایش می‌یابد:

C

```
tl->turn++;
```

C

```
int ticket_acquire(void);
int ticket_release(void);
int ticket_turn(void);
```

برنامه تست

برنامه `tickettest.c` حداقل ۵ فرزند می‌سازد. هر فرزند با فاصله کوتاه درخواست قفل می‌دهد. شماره بلیط در kernel چاپ می‌شود و ورود به critical section باید دقیقاً مطابق شماره بلیط باشد.

اجرای پیشنهادی:

BASH

```
tickettest
```

نمونه خروجی مورد انتظار:

TEXT

```
ticket lock test: FIFO order
child pid 7 requests ticket lock
ticket request: pid 7 ticket 0
ticket enter: pid 7 ticket 0 turn 0
child pid 7 ENTERS ticket 0 turn 0
child pid 8 requests ticket lock
ticket request: pid 8 ticket 1
child pid 9 requests ticket lock
ticket request: pid 9 ticket 2
child pid 7 releases ticket 0
ticket release: pid 7 ticket 0 next-turn 1
ticket enter: pid 8 ticket 1 turn 1
child pid 8 ENTERS ticket 1 turn 1
...
ticket lock test done
```

تحلیل: اگر شماره‌های ورود به critical section به ترتیب 0, 1, 2, 3, 4 باشند، FIFO بودن قفل اثبات می‌شود.

build

BASH

```
make clean
make
make fs.img
```

اجرای xv6 با تعداد CPU متفاوت

در محیط دارای QEMU:

BASH

```
make CPUS=1 qemu-nox
make CPUS=4 qemu-nox
```

اجرای تست‌ها داخل xv6

BASH

```
scounttest 8 10000
pctest
rwtest
tickettest
```

نکات طراحی و فرض‌های پیاده‌سازی

1. نسخه نهایی شمارنده syscall روی per-CPU تنظیم شده است، چون مقیاس‌پذیرترین نسخه است.
 2. برای مقایسه نسخه‌های بخش اول، مقدار SYSCALL_COUNT_MODE در param.h قابل تغییر است.
 3. getcpucount در حالت per-CPU مقدار واقعی هر CPU را برمی‌گرداند. در حالت شمارنده سراسری، مقدار فقط روی CPU0 معنی‌دار گزارش می‌شود.
 4. بافر producer-consumer نهایی blocking است و خطای 1- فقط در حالت killed شدن پردازنده هنگام انتظار برمی‌گردد.
 5. قفل خواننده - نویسنده writer-priority است؛ بنابراین starvation نویسنده رفع می‌شود، اما در بار writer-heavy ممکن است readerها انتظار طولانی داشته باشند.
 6. قفل بلیطی برای نمایش FIFO پیاده‌سازی شده و syscall‌های آن یک قفل global هسته‌ای را از فضای کاربر کنترل می‌کنند.
 7. پیام‌های ticket request/enter/release عمداً داخل kernel با cprintf چاپ می‌شوند تا شماره بلیط تخصیص داده‌شده توسط هسته قابل مشاهده باشد.
- همچنین در بازبینی نهایی، مشکل مشاهده نشدن پردازنده‌های ثانویه در xv6 روی برخی نسخه‌های جدید QEMU/SeaBIOS با اصلاح topology در Makefile حل شد. به جای فرم کوتاه 4 smp - از فرم صریح زیر استفاده شد:

TEXT

```
-smp cpus=4, cores=1, threads=1, sockets=4
```

پس از این اصلاح، xv6 هر چهار CPU را شناسایی کرد:

TEXT

```
cpu1: starting 1
cpu2: starting 2
cpu3: starting 3
cpu0: starting 0
```

خروجی واقعی تست شمارنده syscall در حالت CPUS=4 :

TEXT

```
$ scounttest 16 200000
syscall counter test
children: 16 iterations: 200000
CPU 0 getpid count: 797574
CPU 1 getpid count: 804628
CPU 2 getpid count: 803945
CPU 3 getpid count: 793853
CPU 4 getpid count: 0
CPU 5 getpid count: 0
CPU 6 getpid count: 0
CPU 7 getpid count: 0
Total getpid count: 3200000
Expected count: 3200000
```

تحلیل: چون $3200000 = 16 \times 200000$ و مجموع شمارنده‌های CPU‌های 0 تا 3 دقیقاً 3200000 است، نسخه -per-CPU هم از نظر correctness و هم از نظر توزیع روی چند CPU درست کار کرده است. صفر بودن CPU‌های 4 تا 7 طبیعی است، چون NCPU=8 در xv6 تعریف شده ولی QEMU با 4 CPU اجرا شده است.

خروجی تست قفل بلیطی در همین اجرای چندپردازنده‌ای:

TEXT

```
ticket request: pid 36 ticket 0
ticket request: pid 37 ticket 1
ticket request: pid 38 ticket 2
ticket request: pid 39 ticket 3
ticket request: pid 40 ticket 4
ticket enter: pid 36 ticket 0 turn 0
ticket enter: pid 37 ticket 1 turn 1
ticket enter: pid 38 ticket 2 turn 2
ticket enter: pid 39 ticket 3 turn 3
ticket enter: pid 40 ticket 4 turn 4
ticket lock test done
```

ترتیب ورود دقیقاً مطابق شماره ticket است: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$. بنابراین خاصیت FIFO و نبود گرسنگی در ticket lock برقرار است.

در تست‌های pctest و rwtest در حالت CPUS=4 خروجی console به دلیل چاپ همزمان چند پردازش روی چند CPU ممکن است در سطح کاراکتر interleaved شود. معیار صحت در این تست‌ها خاتمه بدون panic/deadlock، چاپ producer-consumer test done و reader-writer test done، و رعایت invariantهای طراحی است. اجرای تک‌هسته‌ای خروجی خواناتر برای گزارش ترتیبی تولید می‌کند، اما اجرای چهارهسته‌ای برای stress concurrency معتبرتر است.

این پروژه چهار جنبه مکمل از همگام‌سازی در xv6 را نشان می‌دهد:

- race condition روی increment ساده و تفاوت تک‌هسته‌ای/چندهسته‌ای؛
- trade-off بین correctness و scalability در قفل سراسری و per-CPU data؛
- ضرورت تفکیک محافظت از داده مشترک با spinlock از انتظار بلنمدت با sleep/wakeup؛
- اهمیت سیاست‌های fairness در reader-writer lock و ticket lock.

پیاده‌سازی نهایی از نظر build، ساخت image، مسیر dist و اجرای واقعی چندپردازنده‌ای معتبر است. خروجی 4=CPUS نشان داد شمارنده‌های per-CPU روی چهار CPU توزیع شده‌اند و ticket lock ترتیب FIFO را حفظ کرده است.